

Diseño, Plan y Resultados de Pruebas de SIGED-Lampa

Validación del producto generado mediante la fábrica agéntica WEBFORGE

Autor: Leandro Matamoros

Repositorio: LeandroEsteban/Agentes

Versión del producto: v0002

Rama evaluada: main

Commit evaluado: 4dc3e7e08175a469f5f3fd6d59684d213f9ec6e7

Fecha del commit: 2026-07-12 16:43:56 -04:00

Fecha de ejecución: 2026-07-12, 17:43-17:53 -04:00

Producción declarada: <<http://34.226.69.214>>

Estado global: Parcialmente logrado

Índice

1. Resumen ejecutivo
2. Introducción
3. Objetivos
4. Alcance
5. Estrategia de pruebas
6. Ambientes
7. Criterios de entrada y salida
8. Diseño de pruebas
9. Plan de pruebas
10. Casos de prueba
11. Matriz de trazabilidad
12. Ejecución y resultados
13. Resultados de WEBFORGE
14. Resultados del frontend
15. Resultados del backend
16. Cobertura
17. Pruebas E2E
18. Seguridad y permisos
19. Producción EC2
20. Rendimiento
21. Defectos detectados
22. Riesgos y limitaciones
23. Conclusiones
24. Anexos

1. Resumen ejecutivo

Este entregable valida SIGED-Lampa `v0002` y los controles de WEBFORGE con una estrategia escalonada: análisis estático, pruebas Python del runtime, componentes React con Vitest, suites Node.js del backend, verificaciones estructurales de persistencia, preparación E2E y smoke HTTP de producción. La ejecución se hizo en Windows 11 Home Single Language, Python 3.14.6 y Node.js v24.18.0. El baseline Git identificó `README.md` modificado y los artefactos de este entregable sin confirmar; no había cambios de producto antes de ejecutar. WEBFORGE regeneró artefactos rastreados de prueba fuera del alcance y se restauraron inmediatamente a `HEAD`; el estado final no contiene cambios productivos.

Los resultados actuales confirman tres áreas con evidencia directa. WEBFORGE ejecutó `python -m pytest` con **149 aprobadas** en 27,06 s. El frontend aprobó lint, build y **176 pruebas en 14 archivos**; su ejecución de cobertura volvió a aprobar las 176 pruebas y produjo 85,84% de statements y líneas, 76,80% de branches y 67,76% de functions. Las verificaciones Python estructurales de base de datos aprobaron **63 pruebas** en 0,26 s. Estos resultados son recientes y prevalecen sobre reportes históricos cuando miden el mismo aspecto.

La validación integrada de backend no alcanzó el ambiente requerido. La suite unitaria aprobó 5 de 5 casos. Las suites API, integración y smoke iniciaron correctamente, pero el PostgreSQL QA esperado en

127.0.0.1:55432 no estaba disponible. Por ello se registraron 125, 2 y 3 fallos respectivamente, dominados por ECONNREFUSED, respuestas 500 dependientes de persistencia y un health check 503. Son fallos de ejecución reales y no se reinterpretan como aprobados ni como defectos funcionales confirmados: requieren reejecución en QA provisionado. El comando de cobertura c8 se ejecutó sobre sus 135 pruebas y produjo una muestra parcial de 48,93% statements/líneas, 77,77% branches y 9,76% functions; no es aceptable como cobertura integral por los 130 fallos dependientes de QA.

E2E no se ejecutó para no usar producción como entorno de escritura. Playwright está configurado para Chromium y http://127.0.0.1:4173, pero no declara webServer; tanto ese puerto como PostgreSQL QA estaban cerrados. Existe evidencia histórica separada de 10 flujos y 19 casos funcionales aprobados, útil como antecedente, pero no se presenta como ejecución actual. La producción sí recibió smoke de solo lectura: /, /tramites, /login e /intranet/login respondieron HTTP 200, y /api/health devolvió healthy. Esto confirma disponibilidad HTTP y API, no la interacción visual ni autenticación real.

La meta académica de cobertura es 100%; ningún componente con métrica actual llega a ella. Además, WEBFORGE no dispone de pytest-cov en este ambiente, no hay evidencia verificable de pruebas de carga y el producto continúa en HTTP sin evidencia de HTTPS. El estado general es **parcialmente logrado**: el diseño, inventario, pruebas unitarias, componentes, controles de seguridad, validación estructural y smoke están documentados y cuentan con evidencia; la aceptación final exige levantar QA aislado, reejecutar API/integración/smoke/c8/E2E, publicar cobertura completa y ejecutar una prueba de rendimiento reproducible.

2. Introducción

Una fábrica agéntica necesita probar tanto el producto como los controles que gobiernan su generación. En WEBFORGE, las pruebas cubren work orders, gates, políticas, aislamiento y trazabilidad. En SIGED-Lampa, el riesgo se amplía a autenticación, permisos, servicios REST, integridad PostgreSQL, flujos ciudadanos, operación interna y despliegue. El enfoque combina automatización donde existe suite reproducible con verificación segura de producción, sin enviar datos ni ejecutar operaciones destructivas.

3. Objetivos

Objetivo general: evaluar de forma verificable la calidad de SIGED-Lampa v0002 y de los controles WEBFORGE mediante evidencia actual, diferenciando resultados aprobados, fallidos, omitidos y no ejecutables.

Objetivos específicos:

- Inventariar suites, herramientas, dependencias y comandos reales.
- Ejecutar pruebas locales no destructivas y conservar salidas legibles.
- Medir cobertura cuando la herramienta está disponible, sin alterar umbrales ni exclusiones.
- Comprobar precondiciones E2E y evitar usar producción para flujos con escritura.
- Validar disponibilidad HTTP y salud pública con solicitudes de solo lectura.
- Relacionar requisitos, casos, endpoints y evidencia; documentar defectos y brechas.

4. Alcance

Área	Incluida	Resultado de esta ejecución
WEBFORGE	Runtime Python, work orders, gates, políticas y aislamiento	Ejecutado: 149 aprobadas; cobertura no disponible.

Área	Incluida	Resultado de esta ejecución
Frontend	Lint, componentes, páginas, build y Vitest	Ejecutado y aprobado: 176 pruebas.
Backend/API	Unitarias, API, integración, smoke y c8	Unitarias aprobadas; dependientes de QA fallaron por ambiente.
Base de datos	Pruebas estructurales y verificación de esquema	63 estructurales aprobadas; esquema vivo no verificable.
E2E	Flujos Playwright Chromium contra QA	No ejecutado: stack QA no disponible.
EC2	Smoke HTTP de lectura, API health y rutas públicas	Ejecutado: HTTP 200 y health healthy.
Rendimiento	Inventario de carga y métricas	No verificable: sin herramienta ni resultado SIGED actual.

5. Estrategia de pruebas

Nivel	Objetivo	Herramienta	Alcance	Automatizada
Análisis estático	Detectar errores de estilo y tipos	ESLint, TypeScript	frontend/src	Sí
Unitarias WEBFORGE	Validar runtime, normalización, planificación y seguridad	pytest	tests/	Sí
Componentes	Validar UI, estados y llamadas API simuladas	Vitest, Testing Library	frontend/src/test	Sí
API	Validar contratos, RBAC, payloads y ciclos de vida	node:test	backend/tests/api	Sí, requiere QA
Integración	Validar sesión y ciclos con PostgreSQL	node:test	backend/tests/integration	Sí, requiere QA
Smoke	Validar salud, autenticación y contratos mínimos	node:test, curl	QA y EC2	Sí, con ambiente
E2E	Validar flujos de usuario de extremo a extremo	Playwright Chromium	frontend/src/e2e	Sí, requiere QA
Seguridad	Probar credenciales, ownership, RBAC, inyección y asignación masiva	pytest, node:test	Runtime y API	Sí donde hay entorno
Persistencia	Validar estructura, migraciones y restricciones	pytest, script Node	tests/database , PostgreSQL QA	Mixta
Cobertura	Medir código instrumentado	Vitest V8, c8, pytest-cov	Frontend, backend, WEBFORGE	Sí si disponible
Rendimiento	Medir carga y percentiles	No identificado	SIGED-Lampa	No verificable
Producción	Confirmar disponibilidad sin escritura	curl	EC2 pública	Sí, lectura

La prioridad es P0 para autenticación, autorización, health, persistencia y operaciones de documentos; P1 para flujos ciudadanos, expediente, correspondencia y administración; P2 para contenido, presentación, responsive y accesibilidad. Una ejecución aprobada requiere ambiente declarado, comando exitoso y evidencia conservada.

6. Ambientes

Ambiente	Propósito	Componentes	Restricciones
Local de auditoría	Ejecutar suites sin modificar producto	Windows 11, Python 3.14.6, Node 24.18.0	Sin Docker/QA operativo; npm se invoca como <code>npm.cmd</code> por política PowerShell.
QA previsto	API, integración, smoke, E2E y esquema	PostgreSQL 16, backend 3100, frontend 4173, BD 55432	No estaba disponible en esta ejecución.
CI GitHub Actions	Validar, empaquetar y desplegar	Ubuntu, PostgreSQL 16, npm, scripts de release	Solo se consultó en modo lectura.
Producción EC2	Smoke seguro	Nginx y API publicados por HTTP	Solo GET/HEAD; sin login, creación ni modificación.

7. Criterios de entrada y salida

Entrada: commit identificado, dependencias instalables desde lockfiles, configuración de suites inspeccionada, ambiente QA declarado para pruebas que persisten datos, y credenciales de prueba exclusivamente en QA cuando sean necesarias.

Salida aprobada: comando con código cero, todos los casos ejecutados aprobados, evidencia legible, sin secretos, y cobertura contra meta explícita. Para API, integración, smoke y E2E la salida requiere además PostgreSQL QA, migraciones y seeds vigentes. Un puerto cerrado, dependencia ausente o servicio no iniciado se clasifica como **no ejecutable** antes de iniciar la suite; si la suite inicia y falla, se registra como **fallida por ambiente** con su error.

8. Diseño de pruebas

Las particiones de equivalencia distinguen credenciales válidas/inválidas, actor ciudadano/interno/sin permiso, recurso propio/ajeno, estados activos/cerrados/archivados y datos completos/incompletos. Los valores límite cubren paginación, campos obligatorios, fechas de vigencia y límites de payload que los validadores rechazan. Los casos positivos verifican creación, consulta y transición permitida; los negativos comprueban 401/403/404/409, asignación masiva, inyección SQL tratada como dato y cuerpos inválidos.

Las transiciones se diseñan para documentos, expedientes, correspondencia, contenido público y OIRS: no basta crear; se verifica aprobar, cerrar, publicar, archivar o bloquear una transición duplicada. Las pruebas de permisos se separan por ownership ciudadano y RBAC interno. Los componentes cubren loading, vacío, error de red, mensajes accesibles y confirmación de cierre de sesión. E2E enlaza el navegador con API y persistencia, cubriendo solicitud ciudadana, OIRS anónima, documento, revisión, firma simulada, expediente, correspondencia, administración, contenido público y guards. Responsive y accesibilidad se cubren parcialmente mediante componentes y semántica; no existe una ejecución E2E visual actual que permita declararlos aceptados de extremo a extremo.

9. Plan de pruebas

Actividad	Responsable	Herramientas/datos	Dependencia	Criterio
Inventario y baseline	QA senior	Git, glob, configuraciones	Árbol accesible	Rutas y comandos identificados.
WEBFORGE	QA/runtime	pytest, work orders de prueba	Python	Todos los casos con salida guardada.
Frontend	QA frontend	npm, ESLint, Vitest, V8	lockfile	Lint, tests, cobertura y build.
Backend	QA backend	node:test, c8, fixtures	PostgreSQL QA	Suites contra BD disponible.
Persistencia	QA datos	pytest, verify-schema	PostgreSQL QA	Esquema, migraciones y restricciones.
E2E	QA funcional	Playwright Chromium	QA frontend/API/BD	Flujos sin producción.
Producción	QA/SRE	curl	EC2 publicada	HTTP y health de solo lectura.
Cierre	QA senior	Evidencias y matriz	Resultados completos	Limitaciones, defectos y recomendaciones.

Riesgos principales: QA no provisionado, cobertura incompleta, E2E sin `webServer`, ejecución de scripts contra ambiente equivocado, HTTP sin TLS y ausencia de carga. Las mitigaciones son aislar QA, bloquear producción como destino E2E, conservar comandos/salidas y exigir reejecución de los resultados no concluyentes.

10. Casos de prueba

ID	Módulo	Caso	Precondición	Pasos resumidos	Resultado esperado	Tipo	Automatización
WF-01	WEBFORGE	Procesar work order	Python	Ejecutar runtime de prueba	Estado y artefactos trazables	Unitario	Automática
WF-02	WEBFORGE	Validar gates	Work order válido	Ejecutar tests de gates	Gate refleja evidencia requerida	Unitario	Automática
WF-03	WEBFORGE	Rechazar herramienta no permitida	Política cargada	Solicitar herramienta fuera de allowlist	Acción bloqueada	Seguridad	Automática
WF-04	WEBFORGE	Aislar memoria por proyecto	Dos proyectos	Ejecutar pruebas de aislamiento	Sin fuga de memoria/artefactos	Unitario	Automática
WF-05	WEBFORGE	Generar artefactos	Contexto válido	Ejecutar fase de producción	Artefactos y ledger presentes	Unitario	Automática
WF-06	WEBFORGE	Rechazar acción insegura	Política restrictiva	Intentar acción externa	Rechazo y evidencia	Seguridad	Automática
AU-01	Autenticación	Login ciudadano válido	QA con seed	Enviar credenciales QA	Sesión y portal ciudadano	API/E2E	Automática; QA requerida
AU-02	Autenticación	Login interno válido	QA con seed	Enviar credenciales internas	Sesión de intranet	API/E2E	Automática; QA requerida

ID	Módulo	Caso	Precondición	Pasos resumidos	Resultado esperado	Tipo	Automatización
AU-03	Autenticación	Credenciales inválidas	API disponible	Enviar usuario/clave inválidos	Error controlado, sin token	API	Automática; QA requerida
AU-04	Autenticación	Ruta protegida	Sesión ausente	Abrir ruta de portal/intranet	Redirección o bloqueo	Componente/E2E	Automática
AU-05	Autenticación	Permiso insuficiente	Actor sin permiso	Solicitar recurso administrativo	403 o UI bloqueada	API	Automática; QA requerida
AU-06	Autenticación	Logout	Sesión QA	Cerrar sesión y reutilizar token	Token/sesión revocados	Integración	Automática; QA requerida
PU-01	Portal público	Portada	EC2 o QA	Solicitar /	HTML disponible	Smoke	Automática, lectura
PU-02	Portal público	Trámites	EC2 o QA	Abrir /tramites	Ruta pública disponible	Smoke/E2E	Mixta
PU-03	Portal público	Noticias publicadas	QA con datos	Consultar listado público	Solo publicadas/vigentes	API	Automática; QA requerida
PU-04	Portal público	Avisos	QA con datos	Consultar listado	Solo avisos activos	API	Automática; QA requerida
PU-05	Portal público	Calendario	QA con datos	Consultar eventos	Solo eventos futuros válidos	API	Automática; QA requerida
PU-06	OIRS pública	OIRS anónima	QA disponible	Crear y consultar caso	Tracking y estado coherentes	E2E/API	Automática; QA requerida
CI-01	Portal ciudadano	Dashboard	Sesión ciudadana	Abrir portal	Resumen carga y se presenta	Componente/E2E	Automática
CI-02	Portal ciudadano	Solicitud de trámite	Sesión y catálogo	Completar y enviar	Solicitud persistida	E2E/API	Automática; QA requerida
CI-03	Portal ciudadano	Detalle propio	Recurso propio/ajeno	Consultar ambos IDs	Propio visible, ajeno oculto	API	Automática; QA requerida
CI-04	Portal ciudadano	Perfil	Sesión ciudadana	Actualizar datos válidos/inválidos	Validación y persistencia	Componente/API	Automática; QA requerida
CI-05	Portal ciudadano	Notificaciones	Sesión ciudadana	Consultar estado vacío/con datos	Lista o empty state accesible	API/Componente	Mixta
CI-06	Portal ciudadano	OIRS y mensajes	Caso abierto/cerrado	Enviar vacío, válido y cerrado	Bloqueos y mensajes correctos	Componente	Automática
IN-01	Intranet documentos	Crear documento	QA y rol	Crear con campos válidos	Documento creado/auditado	API/E2E	Automática; QA requerida
IN-02	Intranet documentos	Ciclo de revisión	Documento pendiente	Revisar y aprobar	Estado permitido y auditado	API/E2E	Automática; QA requerida
IN-03	Expedientes	Ciclo de expediente	QA y permisos	Crear, asociar, evento, cerrar	Timeline y cierre válidos	API/E2E	Automática; QA requerida
IN-04	Correspondencia	Creación y ruta	QA y permisos	Crear, enrutar y cerrar	Historial consistente	API/E2E	Automática; QA requerida
IN-05	OIRS interna	Gestión de caso	QA y rol	Consultar/cambiar estado	Contrato y permisos válidos	Componente/API	Mixta

ID	Módulo	Caso	Precondición	Pasos resumidos	Resultado esperado	Tipo	Automatización
IN-06	Reportes	Dashboard con filtro	QA y datos	Consultar fechas válidas/inválidas	Agregados y validación	API	Automática; QA requerida
AD-01	Administración	Usuarios	Rol admin	Listar/crear usuario	RBAC y validación	Componente/API	Mixta
AD-02	Administración	Roles/permisos	Rol admin	Asignar/listar	Solo admin autorizado	API	Automática; QA requerida
AD-03	Administración	Departamentos	Rol admin	Crear/editar	Datos válidos y únicos	Componente/API	Mixta
AD-04	Administración	Catálogos	Rol admin	Crear tipo y duplicado	Creación o 409 controlado	Componente/API	Mixta
AP-01	API	Asignación masiva	QA disponible	Enviar atributo no permitido	Payload rechazado	Seguridad	Automática; QA requerida
AP-02	API	Inyección SQL	QA disponible	Enviar patrón SQL	Tratado como dato, sin fuga	Seguridad	Automática; QA requerida
AP-03	API	Error sanitizado	QA disponible	Forzar error controlado	Sin SQL ni stack expuesto	Seguridad	Automática; QA requerida
DB-01	Base de datos	Migraciones	QA disponible	Ejecutar verificación	17 migraciones registradas	Integración	Automática; QA requerida
DB-02	Base de datos	Esquema/restricciones	Código disponible	Ejecutar pytest estructural	Tablas, FK y CHECK definidos	Estructural	Automática
PR-01	Producción	Health API	EC2 publicada	GET /api/health	JSON healthy y HTTP 200	Smoke	Automática, lectura
PR-02	Producción	Rutas públicas	EC2 publicada	HEAD de rutas públicas	HTTP 200	Smoke	Automática, lectura

11. Matriz de trazabilidad

Requisito	Caso de uso	Funcionalidad	Endpoint	Caso de prueba	Resultado actual
WF-R01	Procesar orden	Work order y artefactos	CLI/runtime	WF-01, WF-05	Aprobado por pytest.
WF-R02	Gobernar ejecución	Gates y política	Runtime interno	WF-02, WF-03, WF-06	Aprobado por pytest.
RF-AUTH	Iniciar sesión	Ciudadano e interno	/api/v1/auth/*	AU-01 a AU-06	Componentes aprobados; API QA no concluida.
RF-PUB	Consultar portal	Portada y trámites	Rutas SPA / contenido público	PU-01 a PU-05	Rutas EC2 200; API QA no concluida.
RF-OIRS	Gestionar OIRS	Caso, seguimiento, mensajes	/api/v1/oirs/*	PU-06, CI-06, IN-05	Componentes aprobados; E2E no ejecutado.
RF-CIT	Gestionar solicitudes	Portal ciudadano y perfil	/api/v1/citizen_*	CI-01 a CI-05	Componentes aprobados; API QA no concluida.
RF-DOC	Gestionar documentos	Documento, revisión, firma	/api/v1/documents/*	IN-01, IN-02	API no concluida por QA.

Requisito	Caso de uso	Funcionalidad	Endpoint	Caso de prueba	Resultado actual
RF-EXP	Gestionar expediente	Crear, asociar, timeline, cerrar	/api/v1/expedientes/*	IN-03	API/E2E no concluida por QA.
RF-COR	Gestionar correspondencia	Crear, enrutar, cerrar	/api/v1/correspondence/*	IN-04	API/E2E no concluida por QA.
RF-ADM	Administrar catálogos	Usuarios, roles, departamentos	/api/v1/users/*, catálogos	AD-01 a AD-04	Componentes aprobados; API QA no concluida.
RF-SEC	Proteger recursos	RBAC, ownership y payload	API transversal	AU-05, AP-01 a AP-03	Unitario aprobado; API QA no concluida.
RF-DB	Persistir datos	Migraciones y restricciones	PostgreSQL	DB-01, DB-02	Estructural aprobado; BD viva no verificable.

12. Ejecución y resultados

Suite	Archivos	Ejecutadas	Aprobadas	Fallidas	Omitidas	Duración	Estado
WEBFORGE pytest	5	149	149	0	0	27,06 s	Aprobada
Frontend Vitest	14	176	176	0	0	22,50 s	Aprobada
Frontend V8 coverage	14	176	176	0	0	25,50 s	Aprobada
Backend unit	2	5	5	0	0	1,02 s	Aprobada
Backend API	8	125	0	125	0	6,70 s	Fallida por QA ausente
Backend integración	2	2	0	2	0	1,63 s	Fallida por QA ausente
Backend smoke	2	3	0	3	0	1,16 s	Fallida por QA ausente
BD estructural Python	5	63	63	0	0	0,26 s	Aprobada
E2E Playwright	11	0	0	0	0	No iniciado	No ejecutable
Producción HTTP	5 verificaciones	5	5	0	0	Menos de 2 s	Aprobada, alcance limitado

No se suman los 176 casos repetidos por la corrida de cobertura al total funcional, porque son la misma suite instrumentada. El conteo distinto de casos ejecutados en la corrida primaria es 523: 393 aprobados y 130 fallidos por ambiente QA no disponible.

13. Resultados de WEBFORGE

`python -m pytest` ejecutó 149 pruebas y aprobó todas. La suite cubre runtime, ejecución, normalización, planificación y seguridad de fases. No se informó cobertura porque `pytest-cov` no está instalado; el intento de importación produjo `ModuleNotFoundError`. La ausencia de esa dependencia es una limitación de observabilidad, no una aprobación de cobertura. El arnés regeneró archivos rastreados de prueba y estos se restauraron a HEAD; es una limitación de aislamiento de la suite, no un cambio al producto. Evidencia: [webforge/test-output.txt](#) y [webforge/coverage-not-available.txt](#).

14. Resultados del frontend

`npm.cmd ci` completó sin vulnerabilidades reportadas. ESLint terminó sin advertencias, Vitest aprobó 176 casos de 14 archivos y `tsc --noEmit && vite build` completó correctamente. La cobertura actual V8 supera los umbrales configurados de 70% líneas/statements y 65% branches/functions, pero no alcanza la meta académica de 100%. Playwright no fue ejecutado por falta de stack QA; el archivo de configuración no inicia servidor propio. Evidencia: [frontend/](#) y [cobertura/frontend-coverage-summary.json](#).

15. Resultados del backend

La unidad de autenticación, validadores, token, autorización y ownership aprobó 5 de 5. API, integración y smoke se lanzaron y fallaron por la dependencia PostgreSQL QA no disponible en el puerto configurado. Por transparencia, se preserva el error y no se considera cobertura ni resultados de esas suites como aprobación. El script `verify-schema.js` requiere una base conectada y no se ejecutó contra producción; las pruebas estructurales Python sí aprobaron 63 de 63 sin requerir base viva. Evidencia: [backend/backend-test-output.txt](#).

16. Cobertura

Componente	Statements	Branches	Functions	Lines	Meta	Estado
WEBFORGE	No verificable	No verificable	No verificable	No verificable	100%	No verificable
Frontend actual V8	85,84%	76,80%	67,76%	85,84%	100%	Parcialmente cumple
Backend actual c8	48,93%	77,77%	9,76%	48,93%	100%	No cumple; corrida incompleta

Las métricas miden instrucciones, ramas, funciones y líneas instrumentadas. La ejecución backend no tiene validez como muestra integral porque API, integración y smoke no alcanzaron PostgreSQL QA. Los reportes históricos de `phase7ar6c` indican mejor cobertura bajo un QA disponible, pero se mantienen solo como antecedente y no sustituyen esta ejecución.

17. Pruebas E2E

Playwright configura Chromium, 60 s por prueba, un worker, retry 1 y `baseURL` <http://127.0.0.1:4173> o `E2E_BASE_URL`; no configura `webServer`. La comprobación de precondiciones confirmó que 4173 y 55432 no estaban disponibles. Por seguridad no se inició `npm run test:e2e` contra producción, dado que contiene flujos de creación y cambios de estado. Estado: **No ejecutado**, no fallido. El antecedente `phase7ar6c` registra 10 flujos, 19 casos funcionales y 3 controles de infraestructura aprobados, pero no se declara como resultado actual. Evidencia: [e2e/playwright-output.txt](#) y [e2e/e2e-execution-summary.json](#).

18. Seguridad y permisos

La unidad backend aprobada comprueba hash de contraseña, token válido/manipulado/expirado, validadores estrictos, permisos derivados de base de datos y ownership ciudadano. Las suites API existentes añaden inyección SQL, asignación masiva, JSON inválido, path traversal, transiciones inválidas y sanitización de

errores, pero no pudieron completar su ejecución actual sin QA. En frontend se probaron guards, permisos, redirección, errores de autenticación, mensajes vacíos y bloqueo de OIRS cerrada/cancelada. El cierre de seguridad requiere reejecutar API contra QA y revisar HTTPS/TLS en producción.

19. Producción EC2

La URL evaluada fue `<http://34.226.69.214>`. Los HEAD de `/`, `/tramites`, `/login` e `/intranet/login` respondieron `HTTP/1.1 200 OK` desde Nginx 1.27.5. `GET /api/health` devolvió `{"status":"healthy","service":"sigid-lampa-api"}` y código cero de curl. No se enviaron credenciales, documentos, solicitudes ni OIRS. Este smoke confirma alcance y salud del API; no prueba renderizado visual, autenticación, base de datos, health de base de datos ni funciones protegidas. La última ejecución pública registrada en la evidencia de GitHub Actions fue la #19, asociada a este SHA y concluida exitosamente el 2026-07-12 20:46:33Z. La consulta nueva no fue posible porque el cliente `gh` no está instalado; no se reejecutó ningún workflow. Evidencia: `produccion/production-smoke-output.txt` y `produccion/github-actions-summary.json`.

El workflow `Deploy to EC2` se dispara en push a `main` y manualmente; valida frontend, inventario/migraciones/seeds, backend, archivos de despliegue, empaqueta, despliega, realiza health/smoke y dispone de scripts de rollback. Los pasos backend se ejecutan desde `v0002/backend`, que no tiene manifiesto propio; npm resuelve el manifiesto ascendente de `v0002`, según la ejecución pública exitosa registrada. Esta dependencia de resolución ascendente es un riesgo de mantenibilidad, no un defecto funcional confirmado.

20. Rendimiento

No se identificaron scripts ejecutables ni resultados actuales de k6, Locust, Artillery, autocannon o benchmark para SIGED-Lampa `v0002`. Por lo tanto, usuarios, duración, solicitudes, error rate y p95 son **no verificables**. Evidencia: `rendimiento/performance-summary.txt`. La recomendación es añadir una carga QA reproducible con escenarios de lectura pública, autenticación QA y consulta de expediente, preservando configuración y resumen JSON.

21. Defectos detectados

ID	Defecto	Severidad	Detectado por	Estado	Evidencia
QA-DEF-OIRS-001	Contexto de auditoría OIRS anónima	No registrada	Registro R6C	Corregido; no revalidado actual	<code>phase7a-defect-register.json</code>
QA-DEF-AUTH-001	Login de fixture devolvía 500	No registrada	Registro R6C	Corregido; no revalidado actual	<code>phase7a-defect-register.json</code>
QA-DEF-OIRS-AUDIT-001	Preservación de auditoría OIRS	No registrada	Registro R6C	Corregido; no revalidado actual	<code>phase7a-defect-register.json</code>
QA-DEF-MOCKS-001	Endpoints documentales mock reemplazados por PostgreSQL	No registrada	Registro R6C	Corregido; no revalidado actual	<code>phase7a-defect-register.json</code>
QA-DEF-E2E-06	Desajustes frontend-backend de expediente	No registrada	E2E R6C	Corregido; no revalidado actual	<code>phase7a-defect-register.json</code>

ID	Defecto	Severidad	Detectado por	Estado	Evidencia
QA-DEF-E2E-07	Campos UI de correspondencia no alineados	No registrada	E2E R6C	Corregido; no revalidado actual	phase7a-defect-register.json
QA-DEF-E2E-08	Código administrativo duplicado devolvía 400 en vez de 409	No registrada	E2E R6C	Corregido; no revalidado actual	phase7a-defect-register.json
QA-DEF-E2E-09	Publicar/archivar requería cuerpo completo	No registrada	E2E R6C	Corregido; no revalidado actual	phase7a-defect-register.json
QA-DEF-E2E-10	Selector del portal público no alineado	No registrada	E2E R6C	Corregido; no revalidado actual	phase7a-defect-register.json
QA-ENV-001	PostgreSQL QA no disponible en 55432	Alta para validación, no defecto de producto confirmado	Ejecución actual	Pendiente de ambiente	backend/*-output.txt

No se incluyeron como defectos confirmados las observaciones de navegación ciudadana, perfil, token o contratos mencionadas sin respaldo directo en el registro inspeccionado. Los 9 defectos históricos están marcados como resueltos por su fuente, pero permanecen *no revalidados actualmente* por la ausencia de QA/E2E.

22. Riesgos y limitaciones

- La meta de cobertura 100% no se cumple en frontend ni en la muestra backend; WEBFORGE no tiene métrica actual.
- E2E no se ejecutó porque no existe `webServer` y el stack QA local estaba ausente.
- API, integración, smoke y schema vivo requieren PostgreSQL QA; sus fallos actuales están documentados.
- Producción se publica por HTTP; no hay evidencia de HTTPS/TLS en esta auditoría.
- No existen pruebas de carga ni métricas p95 verificables para la versión.
- El smoke EC2 solo valida HTTP/API health; no prueba procesos de escritura ni visualización de login.
- Datos de QA y pruebas manuales de UX/accesibilidad requieren un ciclo independiente y controlado.

23. Conclusiones

El criterio de diseño y plan de pruebas está **parcialmente logrado**. El proyecto posee una estrategia multicapa, inventario verificable, 41 casos consolidados, trazabilidad por módulo, suites automatizadas, evidencia de resultados recientes y un PDF reproducible. WEBFORGE, frontend y validaciones estructurales entregan señal de calidad positiva; el build y smoke público también son favorables.

La aceptación total no es procedente aún. La falta de PostgreSQL QA impidió confirmar 130 casos backend dependientes de persistencia y bloqueó E2E. La cobertura actual no llega a 100%, no hay cobertura WEBFORGE ni rendimiento actualizado, y producción sigue limitada a HTTP de lectura. La próxima iteración debe provisionar QA aislado, aplicar migraciones/seeds, reejecutar backend/c8/E2E, conservar reportes compactos y crear carga reproducible antes de declarar cierre de calidad.

24. Anexos

Comandos ejecutados

```
python -m pytest

python -c "import pytest_cov"

npm.cmd ci

npm.cmd run lint

npm.cmd test

npm.cmd run test:coverage

npm.cmd run build

node -r tools/documentation/load_qa_env.cjs --test --test-concurrency=1 backend/tests/unit/*.test.js

node -r tools/documentation/load_qa_env.cjs --test --test-concurrency=1 backend/tests/api/*.test.js
backend/tests/phase5b2d/phase5b2d-security.api.test.js

node -r tools/documentation/load_qa_env.cjs --test --test-concurrency=1
backend/tests/integration/*.test.js backend/tests/phase5b2d/phase5b2d-lifecycle.integration.test.js

node -r tools/documentation/load_qa_env.cjs --test --test-concurrency=1 backend/tests/smoke/*.test.js

node backend/scripts/run-tests.js coverage

python -m pytest project/siged-lampa/versions/v0002/tests/database

curl.exe -I http://34.226.69.214/

curl.exe --fail --silent --show-error http://34.226.69.214/api/health
```

Inventario de archivos de prueba

Componente	Tipo	Archivos encontrados	Herramienta	Comando
WEBFORGE	Python	5	pytest	python -m pytest
Frontend	Unitarias/componentes	14	Vitest	npm test
Frontend	E2E	11	Playwright	npm run test:e2e
Backend	Unitarias	2	node:test	node ... unit
Backend	API	8	node:test	node ... api
Backend	Integración	2	node:test	node ... integration
Backend	Smoke	2	node:test	node ... smoke
Base de datos	Python estructural	5	pytest	python -m pytest .../tests/database
Producción	Smoke lectura	0 scripts locales	curl	HEAD/GET explícitos
Rendimiento	Herramientas SIGED	0	No identificada	No aplicable

Evidencias conservadas

Área	Evidencia
WEBFORGE	EVIDENCIAS_PRUEBAS/webforge/test-output.txt
Frontend	frontend/lint-output.txt , test-output.txt , coverage-output.txt , build-output.txt
Backend	backend/backend-test-output.txt y salidas por suite
Cobertura	cobertura/frontend-coverage-summary.json , backend-coverage-summary.json
E2E	e2e/playwright-output.txt , e2e-execution-summary.json
Producción	produccion/production-smoke-output.txt , github-actions-summary.json
Rendimiento	rendimiento/performance-summary.txt

No se añadieron capturas PNG: la ejecución E2E no podía iniciar y el smoke de producción se limitó deliberadamente a HTTP de lectura. Las tablas y extractos de salida enlazados reemplazan capturas inexistentes sin fabricar evidencia visual.